

Summary: Transformer for IceCube Interaction Localization

GitHub repository: <https://github.com/jlvi179/Advanced-Deep-Learning>

What the Notebook Does

This notebook trains a Transformer-based neural network to reconstruct the neutrino interaction position (x, y) from IceCube hit data. It loads the parquet event tables, treats variable-length hit lists as sequences, handles padding and masking, applies self-attention mechanisms via a Transformer Encoder, and evaluates the final position predictions using regression metrics and plots.

Architecture Choices

The main design choices are the sequence processing approach, the Transformer Encoder stack, and the event-level readout:

- **Sequence formatting:** each event consists of a varying number of hits. To process these efficiently in batches, we split the concatenated hit data based on event lengths and use `pad_sequence` to ensure all events in a batch have a uniform sequence length.
- **Input Embedding:** since our input features per hit (time, x, y) are low-dimensional, we map them into a higher-dimensional representation (`d_model`) via a linear projection before passing them into the transformer.
- **Positional Encoding:** positional encoding here is usually not necessary because our data set is not structured in a strict sequential order like text (the hits are fundamentally an unordered set of detections, and any temporal information is already explicitly provided as a ‘time’ feature). However, adding it can sometimes help the model if sequence order implicitly correlates with some structure.
- **Transformer Encoder layers:** we utilize standard PyTorch `TransformerEncoderLayers` (with multi-head self-attention) to learn the relationships between all detected photons in an event simultaneously.
- **Masked Pooling and head:** after the Transformer, we apply a masked mean-pooling across the sequence dimension (strictly ignoring padded tokens) to obtain a single aggregated embedding vector per event. A final linear layer maps this to our 2D target position (x, y) .

Fine points to watch:

- **Masking correctly:** a critical step is computing and applying the boolean `src_key_padding_mask` to ensure that artificial padding tokens do not influence the self-attention scores or the final mean-pooling step.

- **Sequence Length limitations:** standard self-attention scales quadratically with sequence length ($\mathcal{O}(N^2)$). For events with a massive number of hits, memory consumption can quickly become a bottleneck, which requires careful batch size tuning.
- **Event preprocessing:** normalization is essential for both the features (time, x, y) and the target labels (xpos, ypos) so that the network gradients flow and converge smoothly.

In short, the notebook treats each IceCube event as a sequence of detection hits, effectively using a Transformer Encoder to weigh and aggregate the relationships among the hits, producing a precise global prediction for the neutrino interaction position.